

임기호

Software Engineer | Enterprise Platform · AI-enabled Delivery · Automation

파편화된 현업 업무를 하나의 운영 플랫폼으로 전환해 왔습니다. 동작하는 **MVP**를 빠르게 제공하고 현업이 직접 써 보며 요구사항과 예외를 찾게 하는 방식으로 일합니다. 최근에는 **AI**를 설계·구현·검증 과정에 활용해 더 많은 가설을 짧은 주기로 확인하고, **Local LLM**을 실제 문서 처리와 내부 검색에 적용하고 있습니다.

01 이 역할과 맞닿는 두 가지 경험

전사 주요 프로젝트

여러 부서와 외부 시스템이 얹힌 업무를 데이터와 업무 흐름으로 풀어냈습니다. 프론트엔드, 백엔드, 데이터, 배포 환경을 함께 다루며 설계부터 운영 전환까지 맡았습니다.

AI 기반 업무 생산성

AI로 MVP 제작 주기를 줄이고 실제 사용에서 드러난 예외를 다음 설계에 반영합니다. 제품에는 규칙 기반 처리와 Local LLM, 사용자 확인 절차를 업무 위험도에 맞게 조합합니다.

02 KWE Transformation Journey

2022	통합 방향 수립	15종 이상의 프로그램과 이기종 DB를 분석해 항공·해운·재무 중심의 통합 방향을 세웠습니다.
2023-25	운영 플랫폼 구축	레거시 기능과 데이터, 외부 연동을 웹 기반 5개 업무 플랫폼으로 옮겼습니다.
2025-26	개발·운영 기반 정비	NestJS 구조화, Worker 분리, 테스트, CI/CD와 컨테이너 배포 체계를 마련했습니다.
현재	AI 기반 고속 MVP	AI로 구현과 검증 주기를 줄여 현업이 일찍 사용하고 더 빨리 문제를 발견하도록 바꾸고 있습니다.

15종 → 5개

업무 플랫폼 통합

10분 → 5초

수작업 조회 자동화

월 1,200시간

RPA 반복 작업 절감

1,000만 건

데이터 이전·검증

문제를 찾고, 작동하는 시스템으로 확인했습니다

기술부터 고르지 않습니다. 현업이 겪는 문제를 실제 데이터와 업무 흐름으로 좁힌 뒤, 가장 작은 기능을 만들어 검증하고 다음 결정을 내립니다.

- 01

KWE Korea · 2022-현재

통합 운영 플랫폼 전환

Next.js · Node.js · NestJS · PostgreSQL · Prisma · Redis · BullMQ

문제 15종 이상의 사내 프로그램과 3개 DBMS가 부서별로 나뉘어 있었습니다. 같은 데이터를 여러 번 입력했고, 프로그램마다 업무 기준이 달라 변경 영향과 장애 원인을 찾기 어려웠습니다.

판단과 구현 프로그램을 그대로 옮기지 않고 항공·해운 수출입과 재무를 5개 업무 단위로 다시 묶었습니다. 공통 인증과 UI, 데이터 접근, 외부 연동은 모노레포 한곳에서 관리했습니다. Express 백엔드는 NestJS의 도메인 경계로 나누고 배치 작업은 Worker와 BullMQ로 분리했습니다.

검증과 변화 관세청과 화물 사이트 조회는 건당 10분에서 5초 이내로 줄었습니다. 삼성 NERP 입력 RPA는 월 1,200시간 상당의 반복 작업을 덜었고, Oracle 데이터 1,000만 건은 PostgreSQL로 옮긴 뒤 양쪽 값을 교차 검증했습니다.
- 02

AI-assisted development · 2026

AI 에이전트 기반 MVP와 업무 발견

Codex · Claude Code · TDD · Playwright · 업무 설계 문서

문제 현업 요구사항은 처음부터 완성돼 있지 않습니다. 문서와 회의만으로 정리하면 실제 입력 순서, 예외 데이터, 부서마다 다른 판단 기준이 개발 후반에 드러났습니다.

판단과 구현 기능을 작은 수직 단위로 쪼개 AI 에이전트로 설계부터 구현, 테스트, 완료 근거까지 이어서 관리했습니다. 삼성 전세기 업무는 실물 서류로 대조 화면부터 만들었습니다. AEPOS에는 레거시 데이터를 새 화면에 직접 입력하는 E2E harness를 구성했고, OneDrive 문서 자동화는 세 가지 브라우저 접근 방식을 독립 MVP로 먼저 시험했습니다.

검증과 변화 삼성 전세기 대조표에서는 구현 가능한 항목과 현업 기준이 비어 있는 항목, 읽을 수 없는 문서가 일찍 드러났습니다. AEPOS E2E에서는 데이터 마이그레이션 오류처럼 화면을 실제로 움직이기 전에는 보이지 않던 문제를 찾았습니다. MVP가 결과물인 동시에 요구사항을 찾는 도구가 됐습니다.
- 03

Local AI in production workflow · 2026

Local LLM 기반 업무 자동화

Local LLM · OCR · Structured JSON · Vitest · On-premise

문제 수출신고필증처럼 형식이 달라지는 문서는 정규식만으로 의미를 안정적으로 읽기 어려웠습니다. 담당 업무 검색은 자연어 표현이 조금만 달라도 정확한 담당자를 찾지 못했습니다.

판단과 구현 문서는 신고번호별 연속 페이지를 묶어 Local LLM에 전달하고, 업로드 성공과 추출 성공을 분리했습니다. 일부 필드가 비어도 정상 결과는 저장하도록 했습니다. R&R 검색에서는 서버가 찾은 후보 안에서만 LLM이 순위를 바꾸도록 제한해 없는 담당자를 답으로 내놓지 않게 했습니다.

검증과 변화 실제 PDF 반복 평가와 회귀 테스트로 문맥 손실, 응답 흔들림, OCR 오인식을 확인했습니다. 실패하거나 신뢰하기 어려운 결과는 기존 검색을 사용하거나 담당자가 다시 확인하도록 했습니다. AI에 결정을 맡기지 않고 사람이 최종 판단하는 구조(human-in-the-loop)를 원칙으로 삼았습니다.

여러 기술을 거쳤지만, 문제를 푸는 기준은 같았습니다

01 Experience

KWE Korea

2022.01-현재

전략개발부 차장 · Lead Software Engineer

- 15종 레거시 시스템을 5개 업무 플랫폼으로 통합하는 장기 전환을 설계하고 개발·운동을 이끌었습니다.
- 웹·백엔드·DB·RPA·외부 연동·배포 환경을 함께 다루며 부서 간 업무 흐름을 하나의 데이터로 연결했습니다.
- NestJS 구조화, Worker 분리, 테스트와 CI/CD를 도입해 개인 경험에 의존하던 배포와 장애 대응을 표준화했습니다.
- AI 기반 MVP와 Local LLM을 활용해 현업 검증 시점을 앞당기고 문서 처리와 내부 검색 자동화를 진행하고 있습니다.

ILJIN Global

2010.12-2022.01

MES 개발팀 과장 · Software Engineer

- VB6 기반 MES를 C#.NET 서비스로 전환하고 SAP와 생산·재고 데이터를 실시간으로 연계했습니다.
- 제천과 슬로바키아 공장의 MES 고도화를 이끌고 바코드 기반 LOT 추적 체계를 구축했습니다.
- 하루 100만 건 이상의 공정 데이터를 분석해 7가지 품질 기준의 이상 징후를 알리는 SPC 시스템을 개발했습니다.
- 2017년 과장 특진, 2021년 최우수사원 표창을 받았습니다.

02 Technical Foundation

Backend Node.js, TypeScript, Express, NestJS, Prisma, Redis, BullMQ	Frontend Next.js, React, Zustand, React Native, WinForms
Data PostgreSQL, Oracle, MySQL, PL/SQL, PL/pgSQL, 데이터 이전·정합성 검증	Operations GitLab CI/CD, Docker, Docker Swarm, Portainer, PM2, Nginx
AI & Automation Local LLM, WebLLM, OCR, RPA, Puppeteer, Playwright, SAP·관세청·UFS 연동	Stack Adaptability VB6 → C#.NET → Node.js/NestJS로 주력 스택을 전환했습니다. DI, 도메인 경계, 트랜잭션, 비동기 처리, 테스트와 운영 설계는 언어가 바뀌어도 같은 기준으로 적용해 왔습니다.

03 Personal Project

댓글 필터 · Android / Web

2022.01-현재

React Native 앱과 Next.js 웹을 혼자 기획하고 출시했습니다. 웹 버전은 MiniLM 임베딩과 WebLLM을 Web Worker에서 실행해 댓글 분류와 본문 요약 처리합니다. 서버 비용 없이 사용자 데이터를 단말 밖으로 보내지 않는 구조입니다.

Google Play
Web
개발 기록

새로운 코드베이스에 들어가는 방식

언어 문법보다 먼저 업무 흐름과 데이터, 트랜잭션 경계, 실패 지점을 봅니다. 기존 코드의 의도를 테스트로 확인하고 작은 기능을 끝까지 연결한 뒤 범위를 넓혀 왔습니다.